

Orientación a Objetos II

2026

Explicación de práctica
Semana del 11 de mayo



FACULTAD DE INFORMATICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Ejercicios de la semana

En el cuadernillo de Patrones

- Ejercicio 23: Genealogía salvaje
- Ejercicio 24: Acceso bajo demanda
- Ejercicio 25: Planes médicos
- Ejercicio 26: Préstamos Prendarios
- Ejercicio 27: Gestión de paquetes de viaje

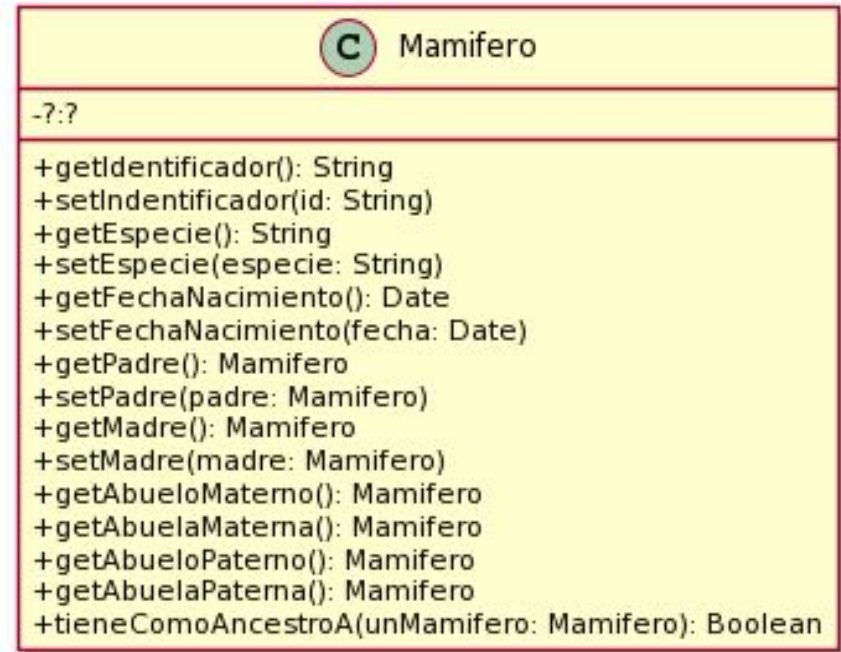
En el cuadernillo de Refactoring: **Refactoring to Patterns**

- Ejercicio 10: Facturación de llamadas
- Ejercicio 11: Árboles binarios
- Repaso -> Ejercicio 12: Triángulo Rectángulo

Ejercicio 23 - Genealogía salvaje

Se trata de una reserva de vida salvaje (como la estación de cría ECAS, en el camino Centenario), donde los cuidadores están interesados en llevar un registro detallado de los animales que cuidan y sus familias. Para ello nos han pedido ayuda.

El siguiente es un diagrama de clases inicial (incompleto) y nos da una idea de los mensajes que un mamífero entiende.



Ejercicio 23 - Genealogía salvaje

¿Como se manejan los casos en donde no se conocen alguno o todos los padres?

- Lo representamos con null
- ¿Se les ocurre hacerlo de otra manera?

```
public boolean madreEs(Mamifero otroMamifero) {  
    return madre != null && madre == otroMamifero;  
}
```

```
public Mamifero getAbuelaMaterna() {  
    if (this.madre != null) {  
        return madre.getMadre();  
    }  
    return null;  
}
```



Mufasa



Sarabi



Simba

Ejercicio 23 - Genealogía salvaje

Null Object Pattern (Woolf & Johnson)

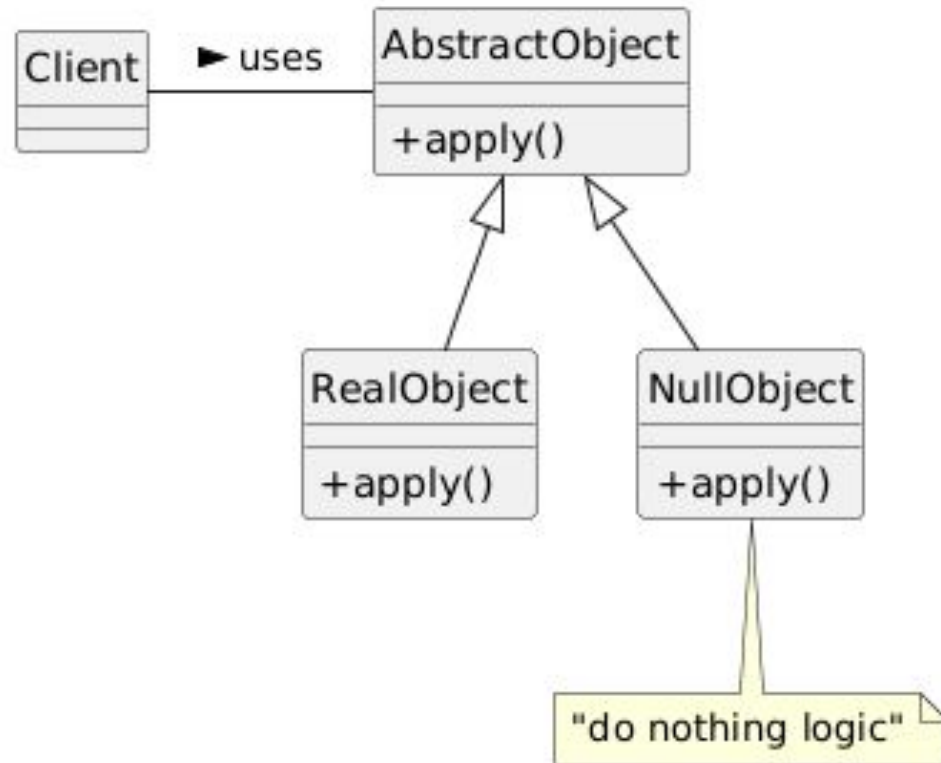
Objetivo: Proporciona un sustituto para otro objeto que comparte la misma interfaz pero no hace nada. El *Null Object* encapsula las decisiones de implementación de cómo "no hacer nada" y oculta esos detalles de sus Colaboradores

Alias: Stub

Consecuencia:

- Elimina todos los condicionales que verifican si la referencia a un objeto es NULL
- Hace explícito elementos del dominio que hacen “nada”

Ejercicio 23 - Genealogía salvaje



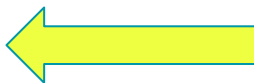
Continuamos el Cuadernillo de Refactoring

|Orientación a Objetos 2

Cuadernillo Semestral de Actividades

-Refactoring -

Actualizado: 7 de mayo de 2026



El presente cuadernillo estará en elaboración durante el semestre y tendrá un compilado con todos los ejercicios que se usarán durante la asignatura respecto al tema Refactoring.

Recomendación importante:

Los contenidos de la materia se incorporan y fijan mejor cuando uno intenta aplicarlos - no alcanza con ver un ejercicio resuelto por alguien más. Para sacar el máximo provecho de los ejercicios, es importante asistir a las consultas de práctica habiendo intentado resolverlos (tanto como sea posible). De esa manera las consultas estarán más enfocadas y el docente podrá dar un mejor feedback.

Refactoring - ¿cómo documentar?

- El código pasa los tests -

Algoritmo de refactoring: Realice en forma iterativa los siguientes pasos

1. Indique el mal olor (su nombre y en qué líneas se encuentra)
2. Indique el refactoring que lo/los corrige (el nombre y los pasos involucrados para realizarlo)
3. Aplique el refactoring, mostrando el resultado final (código y/o diseño según corresponda)
4. Si vuelve a encontrar un mal olor, retorne al paso 1.

Refactoring - Code smell

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
60 public boolean madreEs(Mamifero otroMamifero) {  
61     return this.madre != null && this.madre == otroMamif  
62 }  
63
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
64 public boolean padreEs(Mamifero otroMamifero) {  
65     return this.padre != null && this.padre == otroMamif  
66 }  
67
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
68 public Mamifero getAbuelaMaterna() {  
69     if (this.madre != null) {  
70         return this.madre.getMadre();  
71     }  
72     return null;  
73 }  
74
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
75 public Mamifero getAbuelaPaterna() {  
76     if (this.padre != null) {  
77         return this.padre.getMadre();  
78     }  
79     return null;  
80 }  
81
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
82 public Mamifero getAbueloPaterno() {  
83     if (this.padre != null) {  
84         return this.padre.getMadre();  
85     }  
86     return null;  
87 }
```

Code Smell: Null Check

“Repeating such null logic in one or two places in a system isn’t a problem, but repeating it in multiple places bloats a system with unnecessary code”

Refactoring: Introduce Null Object

- La lógica para manejarse con un valor nulo en una variable está duplicado por todo el código

Introduce Null Object - Mecánica

1. Crear el *null object* aplicando “**Extract Subclass**” sobre la clase que se quiere proteger del chequeo por null (clase origen). **Alternativamente** hacer que la nueva clase implemente la misma interface que la clase origen. Compilar.
2. Buscar un null check en el código cliente, es decir, código que invoque un método sobre una instancia de la clase origen si la misma no es null. Redefinir el método en la clase del null object para que implemente el comportamiento alternativo. Compilar
3. Repetir el paso 2 para todos los null checks asociados a la clase origen.
4. Encontrar todos los lugares que pueden retornar null cuando se le pide una instancia de la clase origen. Inicializar con una instancia del null object lo antes posible. Compilar
5. Para cada lugar elegido en el paso 4, eliminar los null checks asociados

Introduce Null Object - Mecánica

1. Crear el *null object* aplicando “**Extract Subclass**” sobre la clase que se quiere proteger del chequeo por null (clase origen). Alternativamente hacer que la nueva clase implemente la misma interface que la clase origen. Compilar.

```
1 public class NullMamifero extends Mamifero {  
2     public NullMamifero() {  
3         super();  
4     }  
5 }
```

Introduce Null Object - Mecánica

2. Buscar un null check en el código cliente, es decir, código que invoque un método sobre una instancia de la clase origen si la misma no es null. Redefinir el método en la clase del null object para que implemente el comportamiento alternativo. Compilar

```
1 public boolean padreEs(Mamifero otroMamifero) {  
2     return this.padre != null && this.padre == otroMamifero;  
3 }  
4
```

```
1 @Override  
2 public boolean padreEs(Mamifero otroMamifero) {  
3     return false;  
4 }
```

Introduce Null Object - Mecánica

2. Buscar un null check en el código cliente, es decir, código que invoque un método sobre una instancia de la clase origen si la misma no es null. Redefinir el método en la clase del null object para que implemente el comportamiento alternativo. Compilar
3. Repetir el paso 2 para todos los null checks asociados a la clase origen.

Hacemos lo mismo con el resto de los métodos

Introduce Null Object - Mecánica

4. Encontrar todos los lugares que pueden retornar null cuando se le pide una instancia de la clase origen. Inicializar con una instancia del null object lo antes posible. Compilar

```
1 public class Mamifero {  
2     private Mamifero padre;  
3  
4     public Mamifero() {  
5         // el constructor inicializa padre en null ...  
6     }  
7 }
```

Introduce Null Object - Mecánica

4. Encontrar todos los lugares que pueden retornar null cuando se le pide una instancia de la clase origen. Inicializar con una instancia del null object lo antes posible. Compilar

```
1 public class Mamifero {  
2     private Mamifero padre;  
3  
4     public Mamifero() {  
5         this.padre = new NullMamifero();  
6     }  
7 }
```

Introduce Null Object - Mecánica

5. Para cada lugar elegido en el paso 4, eliminar los null checks asociados

```
1 public class Mamifero {
2     public Mamifero getAbuelaPaterna() {
3         if (this.padre ≠ null) {
4             return this.padre.getMadre();
5         }
6         return null;
7     }
8
9     public Mamifero getAbueloPaterno() {
10        if (this.padre ≠ null) {
11            return this.padre.getPadre();
12        }
13        return null;
14    }
15
16    public boolean padreEs(Mamifero otroMamifero) {
17        return this.padre ≠ null && this.padre = otroMamifero;
18    }
19
20    public boolean tieneAncestroPaterno(Mamifero otroMamifero) {
21        return this.padreEs(otroMamifero) || (this.padre ≠ null && this.padre.tieneComoAncestroA(otroMamifero));
22    }
23 }
```


Introduce Null Object - Mecánica

5. Para cada lugar elegido en el paso 4, eliminar los null checks asociados

```
1 public class Mamifero {
2     public Mamifero getAbuelaPaterna() {
3         if (this.padre != null) {
4             return this.padre.getMadre();
5         }
6         return null;
7     }
8
9     public Mamifero getAbueloPaterno() {
10        if (this.padre != null) {
11            return this.padre.getPadre();
12        }
13        return null;
14    }
15
16    public boolean padreEs(Mamifero otroMamife
17        return this.padre != null && this.padr
18    }
19
20    public boolean tieneAncestroPaterno(Mamife
21        return this.padreEs(otroMamifero) || (
22    }
23 }
```

```
1 public class Mamifero {
2     public Mamifero getAbuelaPaterna() {
3         return this.padre.getMadre();
4     }
5
6     public Mamifero getAbueloPaterno() {
7         return this.padre.getPadre();
8     }
9
10    public boolean padreEs(Mamifero otroMamifero) {
11        return this.padre == otroMamifero;
12    }
13
14    public boolean tieneAncestroPaterno(Mamifero otroM
15        return this.padreEs(otroMamifero) || (this.pac
16    }
17 }
```

Ejercicio 27: Gestión de paquetes de viaje

Una agencia de viajes posee un sitio web para gestionar las solicitudes de paquetes de sus clientes. De un paquete de viaje se conoce su precio base, su plataforma de pago y su situación actual (para darle seguimiento). El sistema debe soportar las siguientes operaciones sobre los paquete de viaje:

- **confirmar()** - formaliza la reserva del paquete solicitado si su situación actual lo permite.
- **cancelar()** - anula la reserva del paquete con una penalización que varía según su situación actual.
- **calcularPrecio()** - calcula y retorna el precio efectivo, que varía según la situación actual del paquete.
- **generarResumen()** - produce un texto descriptivo del paquete.

Se presentan dos posibles implementaciones:

Ejercicio 27: Gestión de paquetes de viaje

Tareas:

Responda a las siguientes preguntas:

- a. ¿La variable estado, implementa correctamente el patrón State? Responda si o no.
- b. ¿Respecto del patrón State, es correcto que la misma clase PaqueteViaje implemente el comportamiento a realizar según el estado? Responda si o no.
- c. Realice el diagrama UML del código. Si ud. considera que el patrón State está presente, marque los roles que se encuentran implementados. En caso de que no existen, adecue el diseño para que se implemente correctamente.
- d. ¿La variable estrategia, implementa correctamente el patrón Strategy? Responda si o no.
- e. Explique con sus palabras cual es el objetivo del patrón Strategy

Ejercicio 12: Triángulo rectángulo

Tareas

1.- A partir del código proporcionado, completa los espacios en blanco de la siguiente tabla indicando según corresponda, la(s) línea(s) de código donde aparece el bad smell, el nombre del mismo, justificación que explique por qué es un badsmell y qué técnica de refactoring se emplea para resolverlos.

Bad smell	#Línea(s)	Justificación	Refactoring
Uncommunicative Name			
	#18		
			Remove Parameter

2.- Muestre el código resultante luego de aplicar los refactorings indicados.

